

Digital Wallet System Report

Name – Devpura Dhruvin Vijayprakash

Reg. No. – 25BAI10276

Branch – CSE (Artificial Intelligence and Machine Learning)

Year – Second Year

Course – Programming in Java – CSE2006

Date of Submission – 17 September 2026

University – VIT Bhopal University

Introduction

Digital wallets let people keep, send and check their money using a system. This project builds a version of such a wallet to test how the main functions work in a Java command-line app. The goal is to bring different Java programming ideas into one working system that feels like real business software.

The program starts with a menu where users can register log in or exit. Once logged in they go into the wallet menu to do things like add money make payments or view transactions. The design uses classes to split up tasks. There's a User class for user details Wallet for storing money WalletService to handle actions Transaction for records Beneficiary for sending to others BillPayment for paying bills Database for saving data and DataManager to manage everything.

This separation keeps the core business logic separate, from the input loop. It also lets the app stay simple even though it does things. There's no need to enter a database username or password. The SQLite database runs locally as wallet.db and all user and wallet data gets saved into users.dat. That means anyone can run this app easily on their computer without extra setup.

Objectives

- Develop a practical Java application representing a common business problem.
- Apply encapsulation, inheritance, abstraction, interfaces, and polymorphism in a working system.
- Use custom exception handling to manage invalid wallet operations.
- Use Java collections to store users, transactions, and beneficiaries.
- Demonstrate file I/O through object serialization and statement export.
- Demonstrate JDBC connectivity using a local SQLite database.
- Demonstrate thread synchronization for shared wallet balance operations.
- Provide a simple command-line interface that can be executed from an IDE or command line.

Methodology

The development of the Digital Wallet System followed a plan to create a simple and usable console-based application.

First we identified the needs of the Digital Wallet System. These needs include user registration, PIN-based login, wallet creation, money management, money transfer, beneficiary management, bill payments, transaction history and statement generation.

Next we split the Digital Wallet System into Java classes by responsibility. Classes like User, Wallet, Transaction, Beneficiary and BillPayment were used to represent the entities. WalletService manages wallet operations and DataManager and Database take care of data persistence.

The Digital Wallet System was built with Java object-oriented ideas such, as encapsulation, inheritance, abstraction, interfaces and polymorphism. We added input validation and a custom WalletException to catch input and wallet-related mistakes.

For saving data the Digital Wallet System uses Java file I/O to keep user and wallet information. It uses SQLite with JDBC for database work. We also showed how to use multithreading by adding synchronized methods that protect shared wallet data.

Finally we tested the Digital Wallet System with user actions: registration, login, wallet transactions, bill payments, beneficiary management, transaction history and statement export. After finishing we organized the project files. Uploaded them to GitHub together with the needed documentation and screenshots.

System Features

Feature	Description	Relevant Classes
Registration	Creates a user with name, email, phone number, and four-digit PIN.	Main, User, DataManager, Database
Login	Checks email and PIN against stored users.	Main, User
Wallet Management	Creates a wallet and displays the current balance.	User, Wallet, WalletService
Add / Withdraw Money	Updates the wallet balance and records transactions.	Wallet, WalletService, Transaction
Send Money	Transfers money from one wallet to another and records both sides.	WalletService, Transaction
Beneficiaries	Adds, views, and removes saved beneficiaries.	Beneficiary, WalletService
Bill Payment	Processes electricity, water, mobile, internet, and gas bills.	BillPayment, WalletService, Payment classes
Transaction History	Displays recorded wallet transactions.	Transaction, WalletService, Main
Statement Export	Writes wallet balance and transaction history to a text file.	WalletService, File I/O
Persistence	Stores users and wallet objects locally and basic user records in SQLite.	DataManager, Database

System Design and Class Structure

The project uses a small set of domain classes and service classes. The Main class handles the user-facing menu flow, while WalletService contains wallet business operations. User represents the customer and owns a WalletService instance after a wallet is created.

Core Classes

Class	Responsibility
Main.java	Application entry point, menu flow, input validation, login and registration handling.
User.java	Stores user details and manages the relationship with the wallet service.
Wallet.java	Stores wallet balance and active status and performs balance updates.
WalletService.java	Provides wallet operations such as add money, withdrawal, transfer, beneficiary management, bill payment, and statement export.
Transaction.java	Stores transaction type, amount, description, and date/time.
Beneficiary.java	Stores beneficiary name and email.
BillPayment.java	Stores bill type, consumer number, amount, and payment date/time.
DataManager.java	Persists the user list using Java object serialization.
Database.java	Creates and accesses the SQLite JDBC table for basic user records.
WalletException.java	Custom runtime exception for wallet-related errors.
PaymentMethod.java / BasePayment.java	Define the payment abstraction and common payment type behavior.
ElectricityPayment.java / MobilePayment.java / InternetPayment.java	Concrete payment classes used to demonstrate inheritance and polymorphism.
PaymentProcessor.java	Processes a PaymentMethod reference polymorphically.
WalletConcurrencyDemo.java	Demonstrates synchronized operations on a shared wallet balance.

Basic Flow

- The application initializes the SQLite table and loads the persisted user list.
- The user selects Register, Login, or Exit.
- A logged-in user enters the wallet menu.
- WalletService performs business operations and creates transaction records when applicable.
- DataManager saves user and wallet state after changes and at logout/exit.
- Database stores the basic user record through JDBC.

Database and Persistence

The project demonstrates JDBC using SQLite. The Database class creates a local wallet.db file and initializes a jdbc_users table containing an integer ID, name, and unique email. PreparedStatement is used for inserting user data, while ResultSet is used when reading records.

The use of try-with-resources ensures that database connections, statements, and result sets are closed after use. The application does not store a database username or password because the database is local SQLite.

JDBC Element	Use in Project	Purpose
DriverManager	getConnection()	Opens a connection to wallet.db
Statement	initialize() / showUsers()	Executes fixed SQL commands
PreparedStatement	saveUser()	Inserts values using placeholders
ResultSet	showUsers()	Reads selected database rows

Testing and Result

Test Case	Action	Expected Result	Observed Result
Registration	Enter valid name, email, phone and PIN	Account is created successfully	Pass
Invalid Phone	Enter a non-10-digit phone number	Validation message appears	Pass
Invalid PIN	Enter a PIN other than four digits	Validation message appears	Pass
Invalid Email	Enter an invalid email format	Validation message appears	Pass
Wallet Creation	Create wallet after login	Wallet is created	Pass
Add Money	Add a positive amount	Balance increases and transaction is stored	Pass
Withdraw	Withdraw within available balance	Balance decreases and transaction is stored	Pass
Send Money	Send amount to another wallet	Sender balance decreases and receiver balance increases	Pass
Bill Payment	Pay a selected bill	Amount is deducted and payment is recorded	Pass
Transaction History	Open history after transactions	Recorded transactions are displayed	Pass
Statement Export	Export after transactions exist	Text statement is created	Pass
Persistence	Exit and log in again	Stored user/wallet information remains available	Pass

Advantages

- Simple and user-friendly console-based interface.
- Provides secure 4-digit PIN-based login.
- Allows users to manage wallet balance easily.
- Supports sending money between registered users.
- Provides beneficiary management for saved recipients.
- Supports multiple types of bill payments.
- Maintains transaction history for wallet activities.
- Allows users to export wallet statements as text files.
- Uses file I/O for persistent user and wallet data.
- Uses SQLite with JDBC for database operations.
- Demonstrates object-oriented programming concepts in a practical application.
- Includes exception handling for invalid operations and input.
- Demonstrates multithreading and synchronization for shared wallet operations.
- Modular class structure makes the system easier to maintain and extend.

Disadvantages

- The application is command-line based and does not have a graphical user interface.
- The system uses local storage, so it is not designed for large-scale deployment.
- SQLite is suitable for the project but is not intended for high-volume concurrent transactions.
- The application does not connect to real banking or payment networks.
- Bill payments are simulated and do not actually process real bills.
- The system does not include OTP or biometric authentication.
- The application requires the local Java environment and project files to run.
- The current implementation is designed primarily for educational purposes.

Conclusion

The Digital Wallet System brings together Java ideas into one useful program. The Digital Wallet System shows design by using separate domain and service classes. The Digital Wallet System shows inheritance and interfaces in the payment model. The Digital Wallet System shows exception handling with a custom wallet exception. The Digital Wallet System uses collections by means of lists. The Digital Wallet System uses file input and output by means of serialization and, by exporting statements. The Digital Wallet System uses JDBC with SQLite database operations. The Digital Wallet System demonstrates synchronization with a multithreading example.

The Digital Wallet System also gives a command-line workflow that makes the features easy to test. The Digital Wallet System is intended to be educational, not a real financial program. The Digital Wallet System concentrates on showing programming ideas and how different Java parts work together.

References

- CSE2006 Programming in Java course syllabus, used for the project topic alignment and Java concept coverage.
- Oracle Java Documentation, Java language and standard library reference.
- SQLite JDBC driver documentation / project information for local JDBC connectivity.